Preguntas difíciles — respuestas honestas

Cada respuesta dice qué está verificado y dónde. Si la respuesta es "no sabemos" o "no funciona todavía", se dice así.

¿Por qué no Pharo? Pharo es el entorno de Smalltalk vivo y protoST no compite con él. protoST explora otra cosa: correr código Smalltalk desde archivos, sobre un núcleo (protoCore) con hilos nativos sin lock global y otros lenguajes al lado. Si lo que necesitan es el entorno vivo, Pharo.

¿Dónde está la imagen? ¿Y el IDE? No hay imagen, por diseño: el programa es el archivo. Hay REPL (`protost -i`), debugger de línea de comandos (`protost -d`) y un adaptador DAP para VS Code (`protost --dap`, `docs/debugging.md`). No hay browser de clases.

¿Rendimiento frente a Pharo? No medimos contra Pharo y no vamos a dar una cifra que no medimos. Medimos contra CPython 3.14 con el mismo algoritmo, el mismo N y el mismo resultado verificado (`benchmarks/reports/2026-10-12-protocore-2.14.6.md`: benchmarks sintéticos, en una notebook con un AMD Ryzen 5 5500U de 6 núcleos, con el binario publicado de protoST sobre protoCore 2.14.6 y sobre 2.14.5):

- arranque: 25 ms (CPython: 32–35 ms);
- tiempo de proceso completo: de 0,9× a 21–22× el de CPython, media geométrica 3,2–3,3× en dos corridas; con N chicos el arranque pesa mucho en cada tiempo;
- descontando el arranque de cada runtime, el trabajo en sí tarda unas 4–7× (bucle de enteros), 30–35× (`fib`, despacho recursivo de métodos) y 55–64× (excepciones) lo que tarda CPython; la media geométrica da entre 14× y 20× según la corrida (la resta es sensible al ruido con N chicos);
- en paralelo, doce actores corren 2,8× más rápido con el pool que con un worker, y `saturation_big` llega a 2,5× con cuatro workers y 2,9× con seis, sin techo hasta seis.

Desde 0.6.2 protoST usa el controlador adaptativo del heap de protoCore y dimensiona bajo demanda el arreglo de frames de cada hilo (K6): frente a 0.6.1, los programas de actores van 16–43 % más rápido con entre un quinto y la mitad de la memoria (`benchmarks/reports/2026-10-12-adaptive-heap-default.md`). Buscando una regresión chica en los actores apareció un bug real, corregido en 0.6.1: una recursión sin bucles (el fib de libro) nunca dejaba recolectar su basura; fib 29 se quedaba sin memoria (`benchmarks/reports/2026-10-11-actor-regression-bisection.md`).

Es decir: en un hilo protoST es bastante más lento que CPython. Lo que mostramos es que el mismo código usa varios núcleos.

¿Por qué la demo 2 muestra menos aceleración que antes? Porque el código de un hilo se hizo más rápido. protoCore 2.14.6 dejó de crear temporales al comparar, dividir y sacar el módulo de enteros chicos, y la demo 2 es exactamente eso (`isPrime` en un bucle). Medido con el mismo binario sobre las dos versiones de protoCore, 10 rondas alternadas con la máquina quieta y en modo AC (12/10; medianas):

	protoCore 2.14.5	protoCore 2.14.6	cambio
un hilo	1358 ms	716 ms	47 % más rápido
doce actores	476 ms	293 ms	38 % más rápido
aceleración mostrada	2,8×	2,4×	

(Las cifras anteriores de 4,4× y 3,8× se midieron con la máquina cargada: la carga frena más la mitad de un hilo, que no puede repartirse, y infla la razón.)

Los dos tiempos mejoran; el de un hilo mejora más, y la razón baja. El paralelismo no empeoró: los doce actores terminan antes que nunca. En el resto de los benchmarks 2.14.6 no cambia nada fuera del ruido salvo las excepciones (7 % más rápidas) y `parallel_speedup` con el pool (8 %). El informe de protoCore midió lo mismo en otra sesión (un hilo 33 % y doce actores 26 % más rápidos, razón 2,99× → 2,66×); las cifras de la charla son las de la máquina quieta, arriba.

Antes de 2.14.6 esos temporales hacían depender la demo de la glibc: en la auditoría previa a la charla, la mitad de un hilo iba un 10 % más rápido con la glibc 2.39 de Ubuntu 24.04 que con la 2.43 de la notebook (`benchmarks/reports/2026-10-12-pre-talk-063.md`); con 2.14.6 ya no reservan memoria. Y con la máquina ocupada todas las variantes bajaban a 2–3×: la cifra en pantalla depende de la máquina y de su carga.

¿Cómo es el GC? El de protoCore: concurrente, con un hilo propio. Desde 0.6.2 protoST deja que el controlador adaptativo de protoCore dimensione el heap: un presupuesto del 75 % de la memoria física (o del límite del proceso)

y, dentro, un límite blando que arranca en 128 MB y solo crece mientras el programa espera al recolector. Si la memoria viva no entra en el presupuesto, termina con un mensaje claro. Pero el pico ya no está acotado a 640 MB: un bucle que asigna sin parar llegó a 0,88–3,5 GB en tres corridas (informe del 12/10). Si importa un tamaño fijo, `PROTOCORE_HEAP_LIMIT_CELLS=<celdas>` fija un techo (celdas de 64 bytes: 10000000 son 640 MB de celdas), y `PROTOCORE_ADAPTIVE_HEAP=0` vuelve al techo de 640 MB de 0.6.1 (`KNOWN_ISSUES.md`, K5). Ojo: ese techo cuenta celdas, no la memoria del proceso. Con `PROTOCORE_ADAPTIVE_HEAP=0`, `(1 to: 500000) asArray` se queda sin memoria; con protoCore 2.14.5 el pico de RSS llegaba a 1165 MB (los 1,19 GB de la auditoría: 1 192 800 kB), porque la lista de trabajo del marcado del recolector era un vector que se duplicaba fuera del heap de celdas (hasta 512 MB). protoCore 2.14.6 la acota a una pila fija de 8 MiB: el mismo programa llega a 690 MB y termina en 3,0 s en vez de 5,9 s (informe del 12/10 sobre 2.14.6, cinco corridas). Lo que sobra por encima de las celdas es ahora la lista de celdas marcadas y el resto del proceso. Hasta 0.6.1 no se usaba porque con él `fib.st` tardaba tres veces más; la causa era de protoST (una recursión no entregaba su basura), corregida en 0.6.1. El barrido, la fase más cara, recorre varias cadenas a la vez y puede sumar hilos auxiliares (2.14). Las pausas stop-the-world que medimos en protoST en los benchmarks no pasan de 0,27 ms; antes de 0.6.2 llegaban a 20–31 ms con seis a doce workers, porque cada pausa recorría entero un arreglo de frames de 1 M de entradas por hilo (`KNOWN_ISSUES.md`, K6, resuelto).

¿Por qué prototipos? protoCore es un modelo de prototipos; las clases de protoST son objetos prototipo con nombre. Para el programador Smalltalk se ven como clases (`subclass:`, métodos de clase, variables de clase y de instancia de clase). Las diferencias que eso produce están en el capítulo 14.

¿Con qué se compara el ecosistema protoCore? ¿Cuáles son sus referencias? Sus referencias son conceptuales, no de rendimiento, porque protoCore no optimiza la forma actual de trabajar: intenta cambiar la base. Es un puente entre la programación funcional y los lenguajes mutables.

- **Rich Hickey y Clojure** son la referencia más cercana. Su modelo de identidad, estado y valor dice que una identidad apunta a una sucesión de valores inmutables. Clojure lo ofrece *dentro* de un lenguaje funcional, con `atom` y `ref` explícitos. protoCore lo pone *debajo* de lenguajes mutables: un objeto mutable de Python, JavaScript o Smalltalk es, por construcción, una referencia atómica a una instantánea inmutable, sin que el programador cambie su forma de escribir. El código sigue siendo imperativo; la semántica de fondo es la de valores.
- **Estructuras de datos persistentes** (Okasaki; los árboles de Bagwell): hacen barato que "modificar" cree una versión nueva que comparte casi todo con la anterior.
- **Self y los prototipos de Lieberman:** un modelo de objetos sin clases impuestas, que permite que lenguajes muy distintos compartan objetos.
- **Erlang/BEAM, como contraste:** llega a la concurrencia segura por la vía opuesta, sin compartir nada y copiando los mensajes. protoCore comparte todo sin copiar, porque lo compartido es inmutable.

Lo que resulta es código imperativo de siempre con concurrencia segura, hilos nativos sin lock global y objetos que cruzan de un lenguaje a otro en el mismo proceso, por construcción y no por disciplina.

¿Y GraalVM? ¿Y la JVM? No son comparables en lo que importa. GraalVM es una plataforma de uso general optimizada para los lenguajes existentes, construida por completo sobre el modelo de objetos de la JVM. No propone otra forma de trabajar: optimiza al extremo la actual, y lo hace muy bien. protoCore cambia la base. Si la pregunta es velocidad, ganan ellos sin discusión: protoCore no tiene JIT y no compite en rendimiento de un solo hilo (ver la pregunta de rendimiento).

¿Quién lo usa? Nadie en producción. Es un experimento y un demostrador de protoCore; buscamos gente que lo pruebe con programas reales.

¿Qué falta? `thisContext`, `outer` (hoy se comporta como `pass`), la jerarquía completa de metaclases, `become:`, imagen y browser (por diseño), `Process`, `Semaphore`, `Delay` y `fork` (la concurrencia son actores), clases como `IdentityDictionary`, `ByteArray` o `ScaledDecimal`, y que los runtimes instalados se carguen entre sí (el núcleo ya lo permite desde protoCore 2.6: embebidos en un mismo proceso, protoScala lee y escribe objetos protoST sin copiarlos; `docs/INTEROP.md` §0). Lo que encontramos está en el capítulo 14 del tutorial y en `docs/STATUS.md`; no es una garantía de que no haya otras diferencias.

¿Puede hablar con el mundo exterior? Desde 0.5.0 sí: archivos y directorios (`'datos.csv' asFileReference`), la entrada y salida estándar (`Stdio stdin nextLine`: un script es un filtro Unix), argumentos y variables de entorno, código de salida, otros programas (`OSProcess`), sockets TCP y UDP, TLS, y HTTP cliente (también https) y servidor, con un actor por conexión. Una espera de entrada/salida dentro de un actor no frena al GC y el pool de

workers crece mientras dura, así que muchos actores esperando respuestas no dejan sin hilos a los que las producen. Faltan HTTP/2, WebSockets y un servidor TLS. La demo 4 lo muestra. La capa del sistema operativo es una biblioteca común, protoIO, que protoScala y protoClojure también usan: la misma entrada/salida existe, con los nombres de cada lenguaje, en los tres runtimes.

¿Corre en Windows? El paquete publicado (0.6.5) es para Linux; en Windows funciona con WSL2 y Ubuntu 24.04, instalando los mismos paquetes `.deb`. Las fuentes compilan en forma nativa con Visual Studio 2022 y la integración continua corre la suite en Windows: 1097 casos, todos salvo los que dependen de herramientas POSIX (Ctrl-C enviado con `kill`, un ejemplo con sintaxis de `sh`) y el test de la consola de terminal; `docs/INSTALLATION.md`, "Windows (MSVC)". Todavía no hay un paquete de Windows publicado.

¿Es estable? ¿Qué bugs abiertos tiene? 0.5.0 pasaba sus 1068 casos de `ctest`; 0.6.5 (con protoCore 2.14.6) pasa sus 1101 casos en Linux: programas de conformidad, tests unitarios, los ejemplos, tests de la línea de comandos y los ejemplos de la documentación. La integración continua corre 1100 en Linux y en macOS (más una prueba de paralelismo medida con reloj, que en un runner compartido no bloquea) y 1097 en Windows. Son tests, no los programas de la auditoría: durante la preparación de esta charla una auditoría adversarial de unos 1100 programas, en dos rondas, encontró resultados incorrectos silenciosos y cuelgues; cada corrección de un resultado incorrecto, un crash o un cuelgue tiene su test de regresión. El cuelgue intermitente S19 se capturó con el proceso vivo y resultó ser un despertar perdido en `std::counting_semaphore` de libstdc++ 13; se reemplazó. Su test quedó fuera del gate de la CI mientras se acumulaba evidencia: pasó al primer intento en las 59 corridas de CI desde la corrección y 150 veces seguidas en local, y desde 0.6.3 vuelve a bloquear la CI. Y en 0.6.3 apareció, gracias a una advertencia del compilador, un defecto latente: el despacho con goto computado no liberaba los pins transitorios del recolector, y en los builds de Linux con GCC esos pins dejaban de proteger objetos después de cargar el kernel; está corregido, con su test. Lo abierto está en `docs/STATUS.md`.

¿Los actores pueden bloquearse entre sí? Un actor que hace `wait` dentro de un método no atiende otros mensajes hasta recibir la respuesta. Si dos actores se esperan mutuamente nadie avanza: protoST lo detecta y señala un `Error` ("deadlock") en lugar de colgarse. Dentro de actores conviene encadenar con `thenDo:` o `whenAll:`.

¿Licencia? MIT.

¿Cómo contribuyo? Escribir programas como los escribirían ustedes y reportar lo que sorprenda (issues en el repositorio). `docs/ROADMAP.md` lista el trabajo abierto.